

Programming Assignment 2

Due at the beginning of your discussion session on
September 8-11, 2026

Reading

In addition to the following topics, the quiz syllabus includes any material covered in the lectures:

- Chapter 14 in Code Complete
- Items 28, 60, 62, and 64 in Effective Java
- Java's BigInteger and BigDecimal documentation (introduction only)
- Section 19.1 in Code Complete (excluding "Forming Boolean Expression Positively", "Guidelines for Comparison to 0", "In C-derived languages ...", "In C++, consider ...")
- Section 15.1 ("Plain if-then Statements" only) in Code Complete
- Sections 17.1, 18.1 (excluding "Two Issues ..."), and 19.4 in Code Complete ("Convert a nested if ...", "Factor deeply nested code ...", and "Use a more object-oriented approach" only)
- The example of an object factory on canvas

Programming

The objective of this and the next few programming assignments is to develop a system for fast matrix multiplication.

Matrix multiplication is a fundamental operation in linear algebra, scientific computing, machine learning, and large language models. The tremendous need for matrix multiplication has spawned the

creation of specialized hardware, data centers, and algorithms. In this project, you will implement Strassen's algorithm for matrix multiplication. Strassen's algorithm aims at minimizing the number of floating-point operations, thus leading to a fast algorithm on conventional CPU hardware.

Matrix and Matrix Operations

For this project, a *matrix* is a square table of floating-point numbers. The *size* of a matrix is the number of rows or columns in the matrix. For example, the following is a matrix of size 2:

$$a = \begin{pmatrix} 1 & 2 \\ 3 & 4 \end{pmatrix}$$

The matrix entries are denoted by indexing the matrix name by its row and column number. In the example above, the matrix entries are:

a[1][1]=1	a[1][2]=2
a[2][1]=3	a[2][2]=4

A matrix is *sparse* if most of its elements are 0.

A matrix can be represented as blocks or submatrices pieced together. For example, the matrix

$$\begin{pmatrix} 1 & 2 & 3 & 4 \\ 5 & 6 & 7 & 8 \\ 6 & 7 & 8 & 9 \\ 5 & 4 & 3 & 2 \end{pmatrix}$$

can be represented as

$$\begin{pmatrix} A & B \\ C & D \end{pmatrix}$$

where

$$A = \begin{pmatrix} 1 & 2 \\ 5 & 6 \end{pmatrix}, B = \begin{pmatrix} 3 & 4 \\ 7 & 8 \end{pmatrix}, C = \begin{pmatrix} 6 & 7 \\ 5 & 4 \end{pmatrix}, D = \begin{pmatrix} 8 & 9 \\ 3 & 2 \end{pmatrix}$$

The *addition* of two matrixes of the same size is another matrix whose elements are the sum of those in the original matrixes. For example,

$$\begin{pmatrix} 1 & 2 \\ 3 & 4 \end{pmatrix} + \begin{pmatrix} 4 & 3 \\ 1 & 2 \end{pmatrix} = \begin{pmatrix} 5 & 5 \\ 4 & 6 \end{pmatrix}$$

The *product* of two matrixes a and b of the same size n is denoted as $c = a \cdot b$ and is defined by the formula:

$$c[i][j] = a[i][0] b[0][j] + a[i][1] b[1][j] + \dots + a[i][n] c[n][j]$$

where n is the number of elements in each row and in each column. For example,

$$\begin{pmatrix} 1 & 2 \\ 3 & 4 \end{pmatrix} \cdot \begin{pmatrix} 4 & 3 \\ 1 & 2 \end{pmatrix} = \begin{pmatrix} 6 & 7 \\ 11 & 17 \end{pmatrix}$$

It turns out that matrix multiplication can also be represented in block form assuming that matrixes can be portioned in blocks of size $(n/2) \times (n/2)$ as

$$\begin{pmatrix} A & B \\ C & D \end{pmatrix} \cdot \begin{pmatrix} E & F \\ G & H \end{pmatrix} = \begin{pmatrix} I & J \\ K & L \end{pmatrix}$$

where

$$I = A \cdot E + B \cdot G$$

$$J = A \cdot F + B \cdot H$$

$$K = C \cdot E + D \cdot G$$

$$L = C \cdot F + D \cdot H$$

Strassen's algorithm computes the multiplication of two matrices using a procedure different than the ones above to reduce the number of floating-point operations. Strassen's algorithm works by recursion on the matrix size by subdividing a matrix in progressively smaller blocks and multiplying them. Strassen's algorithm will be the topic of a future programming assignment because a Java representation of sparse matrixes must be implemented first.

Package

You should organize your project in a package. The package name is up to you: it could range from the simple ('strassen') to the detailed ('edu.cwru.<cwruid>.strassen').

Coordinates

The *coordinates* of a matrix entry are the row and column index of that entry. Since locating matrix entries is fundamental to all matrix operations, coordinates are a crucial building block in a sparse matrix representation. Define a public record `Coordinates` with two

integers that represent the row and the column. The `Coordinates` come with the following public methods:

- `negated` returns new `Coordinates` obtained by changing the sign of row and column. For example `negated(1, 1)` returns the coordinates `(-1, -1)`.
- `plus(Coordinates offset)` returns new `Coordinates` obtained by adding the row and column indexes of this `Coordinates` to the other's.
- `minus(Coordinates origin)` is like `plus` but subtracting the indexes.
- `times(int scale)` returns new `Coordinates` obtained by multiplying the row and column indexes of this `Coordinates` by the scale. For example, `times(3)` applied to `(1, 2)` returns the coordinates `(3, 6)`.

A few `Coordinates` are especially useful and should be defined as constants:

<code>ORIGIN</code>	<code>(0,0)</code>
<code>HORIZONTAL_UNIT</code>	<code>(1,0)</code>
<code>VERTICAL_UNIT</code>	<code>(0,1)</code>
<code>DIAGONAL_UNIT</code>	<code>(1,1)</code>
<code>NEGATIVE_HORIZONTAL_UNIT</code>	<code>(-1,0)</code>

`Coordinates` can be compared with each other by row index and breaking ties by column. For example, `(1, 0)` is greater than `(0, 0)` because the row index 1 of `(1, 0)` is greater than the row index of `(0, 0)`. Similarly, `(1, 0)` is greater than `(0, 1)` because the comparison uses the horizontal coordinate first. Furthermore, `(0, 1)` is greater than `(0, 0)` because the tie in the horizontal coordinate is resolved by the vertical coordinate. To implement `Coordinates` comparison, `Coordinates` should implement the `Comparable` interface and define a corresponding public constant `Comparator COMPARATOR`.

Error Handling

Exceptions and Assertions

Error checking will be a topic of a future lecture. For the time being, the following guidelines should be implemented:

- If a method is *visible* outside the package (e.g., public methods in public classes), failed pre-conditions should cause the code to throw an appropriate exception. An exception can carry supplemental information that is passed as the constructor's argument(s).
- If a method is *invisible* from outside the package (e.g., private, or package-private methods), failed pre-conditions should be verified by an `assert`. An `assert` can specify supplemental information as the expression following the colon.

Supplemental Information

In both cases, the error handling should be supplemented by additional information. In Java, the supplemental information can take the form of one or more of the following:

- *Nothing*, if the source of the error is clear from other information already available (such as a stack trace).
- A `String` that outlines the reasons for the error. Strings should only be used if their usage conforms to Item 62 in the reading assignment.
- A `Throwable` that describes the cause of the error.

Null Arguments

Common occurrences of failed pre-conditions are arguments that are *null* even though null parameters are not expected or meaningful. Unless stated otherwise, you should assume through this series of assignments that null parameters constitute a failed pre-condition that necessitates error handling.

The Java Language Features module on canvas contains pointers on assertions, exceptions, and null parameter checking.

Entries

An `Entry` is a public generic record that models a single matrix item. It consists of `Coordinates` and a generic value at those coordinates. `Entry` should have a method

```
Entry<T> translated(Coordinates offset)
```

that returns a new entry with the same value placed at coordinates obtained by adding the offset to the current coordinates.

Sparse Matrix Representation

Since a sparse matrix consists mostly of zero entries, an efficient representation involves recording only the location and values of non-zero entries. The public final generic class `EntryMap` defines the sparse matrix representation. It contains a private final `NavigableMap` called `entryMap` that associates `Coordinates` to generic values. For example, the matrix

$$\begin{pmatrix} 1 & 2 \\ 3 & 4 \end{pmatrix}$$

is represented by the map

Key	Value
(0, 0)	1
(0, 1)	2
(1, 0)	3
(1, 1)	4

Although the generic value can in principle be any type, this project will mostly consider entries that are either floating-point numbers or matrixes (in block representations).

The *private* constructor assigns `entryMap` to the argument and the

```
public static method EntryMap<T>  
from(Map<Coordinates, T> entryMap)
```

returns a new `EntryMap` made from a copy of the argument. The public methods

```
T get(Coordinates coordinates)
```

```
T getOrDefault(  
    Coordinates coordinates,  
    T defaultValue)
```

return the values in the given coordinates. The difference between the two is when there is no entry in the given coordinates. Then, `get` returns `null` and `getOrDefault` returns the `defaultValue`.

The public methods `rows` and `columns` return the largest row and column with non-zero entries (plus one). For example, the matrix

$$\begin{pmatrix} 1 & 2 \\ 0 & 0 \end{pmatrix}$$

has one row and two columns. The public size of the `EntryMap` is the largest of rows and columns.

General Considerations

You should write JUnit tests to make sure that your primary methods work as intended. However, we will revisit testing later in the course, so extensive testing is not yet recommended. If you have never used JUnit, the JUnit module on canvas lists resources to get you started. Similarly, your code should have a reasonable number of comments, but documentation is going to be the topic of a future assignment. As a general guideline at this stage of the course, comments and tests should be like those accepted in CSDS 132. Additionally, comments should only be applied to the code sections that you feel are not self-documenting.

Canvas Resources

The module on Java Language Features contains a folder on useful Java features, such as enumerated types, regular expressions, and the optional class. A discussion board can be used to ask questions and post answers on this programming assignment.

Discussion Guidelines

The discussion will focus on:

- High-level code organization
- The design and documentation of the implementation
- Straight-line code, conditional code

Your grade will be affected by the topics covered in this and in previous weeks. Your discussion leader may introduce material that has not been covered yet in the lecture. Although future topics will not contribute to your current grade, you are expected to follow your leader's advice in revising the assignment.

Submission

Submission instructions will be posted shortly.